



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

título del TFG



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 29 de marzo
de 2026

Tutores: Jose Luis Garrido Labrador y Jose
Miguel Ramirez Sanz



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Álvaro Pérez Mella, con DNI dni, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 29 de marzo de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. nombre tutor

D. nombre co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
3. Conceptos teóricos	5
3.1. Secciones	5
3.2. Referencias	5
3.3. Imágenes	6
3.4. Listas de items	6
3.5. Tablas	7
4. Técnicas y herramientas	9
4.1. Herramientas fase de pruebas	9
5. Aspectos relevantes del desarrollo del proyecto	15
5.1. Enfoque general y ciclo de vida	16
5.2. Planteamiento inicial del problema	17
5.3. Elección tecnológica: Flask y ecosistema Python	17
5.4. Selección de IAs para la demo: por qué YOLO y MediaPipe	18
5.5. Construcción de la demo: objetivos y aprendizajes clave . . .	20
5.6. Decisiones de diseño relevantes en la demo	28
5.7. Aspectos relevantes de implementación en la demo	30

5.8. Limitaciones conocidas de la demo y necesidad de evolución	31
5.9. De la demo a la aplicación real: cambio de enfoque	31
5.10. Análisis de requisitos de la aplicación completa	32
5.11. Del sistema completo al producto mínimo viable	33
5.12. De los casos de uso al diseño de datos	34
5.13. Migración del entorno de desarrollo a Debian	35
5.14. Implementación de la base de datos en MariaDB	37
5.15. Valor de esta transición dentro del desarrollo del TFG	38
6. Trabajos relacionados	41
7. Conclusiones y Líneas de trabajo futuras	43
Bibliografía	45

Índice de figuras

3.1. Autómata para una expresión vacía	6
5.1. Resultado de la ejecución de MediaPipe en modo hands , con representación de landmarks y conexiones sobre ambas manos detectadas.	21
5.2. Interfaz de la demo durante una ejecución de MediaPipe en modo pose , mostrando la imagen de entrada y los landmarks corporales detectados.	23
5.3. Salida JSON generada por MediaPipe en modo pose , con información de landmarks, visibilidad y ruta de la imagen procesada.	24
5.4. Interfaz de la demo con ejecución del modelo YOLO en modo default , mostrando la imagen procesada con cajas delimitadoras sobre los objetos detectados.	26
5.5. Salida JSON asociada a la ejecución de YOLO, incluyendo detecciones, confianza, coordenadas y ruta de la imagen generada.	27

Índice de tablas

3.1. Herramientas y tecnologías utilizadas en cada parte del proyecto	7
4.1. Comparación general entre MediaPipe y YOLO	11

1. Introducción

Descripción del contenido del trabajo y del estructura de la memoria y del resto de materiales entregados.

2. Objetivos del proyecto

Este apartado explica de forma precisa y concisa cuales son los objetivos que se persiguen con la realización del proyecto. Se puede distinguir entre los objetivos marcados por los requisitos del software a construir y los objetivos de carácter técnico que plantea a la hora de llevar a la práctica el proyecto.

3. Conceptos teóricos

En aquellos proyectos que necesiten para su comprensión y desarrollo de unos conceptos teóricos de una determinada materia o de un determinado dominio de conocimiento, debe existir un apartado que sintetice dichos conceptos.

Algunos conceptos teóricos de L^AT_EX ¹.

3.1. Secciones

Las secciones se incluyen con el comando `section`.

Subsecciones

Además de secciones tenemos subsecciones.

Subsubsecciones

Y subsecciones.

3.2. Referencias

Las referencias se incluyen en el texto usando `cite` [[11](#)]. Para citar webs, artículos o libros [[6](#)], si se desean citar más de uno en el mismo lugar [[1](#), [6](#)].

¹Créditos a los proyectos de Álvaro López Cantero: Configurador de Presupuestos y Roberto Izquierdo Amo: PLQuiz

3.3. Imágenes

Se pueden incluir imágenes con los comandos standard de $\text{\LaTeX}$, pero esta plantilla dispone de comandos propios como por ejemplo el siguiente:



Figura 3.1: Autómata para una expresión vacía

3.4. Listas de items

Existen tres posibilidades:

- primer item.
- segundo item.

1. primer item.
2. segundo item.

Primer item más información sobre el primer item.

Segundo item más información sobre el segundo item.

■

Herramientas	App	AngularJS	API REST	BD	Memoria
HTML5		X			
CSS3		X			
BOOTSTRAP		X			
JavaScript		X			
AngularJS		X			
Bower		X			
PHP			X		
Karma + Jasmine		X			
Slim framework			X		
Idiorm			X		
Composer			X		
JSON		X	X		
PhpStorm		X	X		
MySQL				X	
PhpMyAdmin				X	
Git + BitBucket		X	X	X	X
MikTeX					X
TeXMaker					X
Astah					X
Balsamiq Mockups		X			
VersionOne		X	X	X	X

Tabla 3.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

3.5. Tablas

Igualmente se pueden usar los comandos específicos de $\text{\LaTeX}$ o bien usar alguno de los comandos de la plantilla.

4. Técnicas y herramientas

En este capítulo se describen las herramientas principales empleadas o consideradas durante la fase de pruebas del proyecto, centradas en el ámbito del procesamiento de imágenes y la visión por computador. Se presentan de forma general sus características, finalidad y fuentes oficiales de documentación.

4.1. Herramientas fase de pruebas

Durante la fase de pruebas del proyecto se han evaluado diversas herramientas para el procesamiento y análisis de imágenes. A continuación se describen las dos más relevantes: **MediaPipe** y **YOLO**, ambas ampliamente utilizadas en el campo de la visión artificial.

MediaPipe

MediaPipe es un marco de trabajo de código abierto desarrollado por Google para la creación de *pipelines* o flujos de procesamiento multimedia en tiempo real [7, 4, 5].

Permite construir sistemas modulares basados en grafos donde cada componente (denominado *calculator*) realiza una tarea específica, como la captura de imágenes, el preprocesamiento, la inferencia de modelos de aprendizaje automático o el postprocesado de resultados. Su diseño facilita la reutilización de módulos y la integración de modelos de visión por computador en distintas plataformas (Android, iOS, escritorio o web).

Entre las soluciones más utilizadas que ofrece se incluyen:

- Detección de rostro (*Face Detection*) y malla facial 3D (*Face Mesh*).
- Detección y seguimiento de manos (*Hand Tracking*).
- Estimación de pose corporal completa (*Pose*).
- Solución integral *Holistic*, que combina rostro, manos y cuerpo.

Estas funcionalidades permiten aplicar MediaPipe a proyectos de reconocimiento facial, análisis de movimiento, interacción gestual, realidad aumentada y aplicaciones educativas o deportivas.

Toda la documentación oficial se encuentra disponible en la guía técnica de Google AI Edge [4], y el código fuente está publicado en su repositorio oficial de GitHub [5].

YOLO (You Only Look Once)

YOLO es una familia de modelos de detección de objetos en tiempo real creada por la empresa Ultralytics. Su principio básico consiste en dividir una imagen en una cuadrícula y predecir simultáneamente las coordenadas y la clase de los objetos presentes en cada región [10, 9].

La versión más reciente, YOLOv8, incorpora arquitecturas modernas optimizadas para precisión y velocidad, con soporte para varias tareas de visión artificial:

- Detección de objetos (*object detection*).
- Segmentación de instancias (*instance segmentation*).
- Clasificación de imágenes.
- Estimación de pose humana.

Se distribuye como paquete Python mediante la librería *ultralytics*, permitiendo la carga de modelos pre entrenados o el entrenamiento personalizado sobre conjuntos de datos propios.

La documentación y ejemplos de uso están disponibles en el sitio oficial de Ultralytics [10], mientras que el código fuente se mantiene en su repositorio público de GitHub [9].

Aspecto	MediaPipe	YOLO
Enfoque principal	Procesamiento de características humanas (rostro, manos, pose).	Detección y localización de objetos genéricos en imágenes o vídeo.
Tipo de modelo	Framework modular con soluciones preentrenadas de Google.	Red neuronal convolucional profunda (CNN) optimizada para detección.
Ejecución	En tiempo real, incluso en dispositivos móviles o CPU.	En tiempo real, preferentemente con GPU para rendimiento óptimo.
Configuración	Bajo nivel de configuración: soluciones listas para usar.	Permite entrenamiento y ajuste fino sobre datasets personalizados.
Precisión en personas	Alta precisión en puntos clave del cuerpo humano.	Alta precisión en la localización de personas y objetos.
Lenguaje y soporte	Implementación principal en Python y C++.	Implementación en Python con soporte mediante la librería Ultralytics.
Licencia	Apache License 2.0 (Google).	AGPL-3.0 License (Ultralytics).
Uso ideal	Análisis de movimiento, reconocimiento facial, interfaces gestuales.	Detección general de objetos, vigilancia, control de tráfico o inventario.

Tabla 4.1: Comparación general entre MediaPipe y YOLO

Comparativa entre MediaPipe y YOLO

Aunque ambos frameworks se utilizan en el ámbito de la visión por computador, su enfoque y finalidad son distintos y, en muchos casos, complementarios. La tabla siguiente resume sus principales diferencias:

En resumen, **MediaPipe** resulta más adecuado para aplicaciones centradas en la estimación de características humanas y análisis del movimiento, mientras que **YOLO** se utiliza para la detección y clasificación de objetos en escenas más amplias. En conjunto, ambas herramientas pueden integrarse en sistemas híbridos donde se necesite combinar información semántica (qué objetos hay) y estructural (cómo se mueven o se relacionan las personas).

Framework de servidor: evaluación y elección

Para la capa de servidor del proyecto se necesita un API HTTP que reciba imágenes y parámetros, coordine los módulos de IA desarrollados

en Python y asegure que podamos añadir después funciones de seguridad (autenticación, trazabilidad, CORS). Bajo estos objetivos, se han analizado tres frameworks.

FastAPI (Python). **Ventajas:** permite programación asíncrona, genera automáticamente documentación interactiva, asegura tipos de datos con Pydantic. **Inconvenientes:** añade más estructura y definición de datos de la necesaria al inicio. **Motivo de descarte:** se buscaba avanzar rápido con una API sencilla y sin tanta formalidad estructural.

Django REST Framework (Python). **Ventajas:** completo para proyectos grandes, con gestores, serializadores y autenticación ya integrada. **Inconvenientes:** demasiado pesado para un backend cuyo foco principal es orquestar inferencias de IA. **Motivo de descarte:** la complejidad no se ajusta al alcance de este TFG.

Express.js (Node.js). **Ventajas:** minimalista, gran comunidad. **Inconvenientes:** implicaría usar otro lenguaje además de Python, lo que complica la integración con los modelos de IA. **Motivo de descarte:** se opta por mantener todo el servidor en Python para mayor coherencia.

Framework elegido: Flask

Descripción. Flask es un microframework web en Python. Se caracteriza por ofrecer lo esencial —rutas, peticiones, respuestas— sin imponer una estructura compleja. Su diseño ligero permite añadir sólo lo que el proyecto necesite.

Ventajas para este proyecto.

- Permite lanzar rápidamente un endpoint que reciba la imagen y devuelva un resultado, sin código innecesario.
- Al estar en Python facilita conectar directamente con los módulos de IA.
- Su simplicidad permite que, cuando lo necesitemos, añadamos seguridad, control de acceso o logging sin tener que rehacer la arquitectura.
- Los desarrollos iniciales son rápidos y el despliegue sencillo.

Justificación de elección. Dado que el servidor funciona como una capa de orquestación —no como un sistema transaccional complejo— se requiere algo ligero, sencillo de mantener y que permita crecer cuando haga falta. Flask cumple claramente esos requisitos: arranque rápido, bajo coste inicial, buena integración con Python y evolución controlada.

Funcionamiento en el proyecto. Se definirán rutas HTTP para subir imágenes o enviar parámetros, dichas rutas delegarán el trabajo a los módulos internos de IA y devolverán respuestas en formato JSON. La estructura del proyecto podrá separarse en rutas / servicios / módulos, lo que mantiene el código ordenado y escalable.

Librería para simulación de detecciones: evaluación y elección

Para validar tempranamente el *pipeline* del backend (subida de imagen → procesamiento simulado → respuesta con imagen anotada + JSON), se requiere una librería ligera que permita abrir imágenes, dibujar anotaciones sintéticas (cajas, texto) y devolver resultados sin integrar todavía modelos de IA reales. Bajo estos objetivos, se han analizado tres alternativas.

OpenCV (Python). **Ventajas:** muy completa para visión por computador; dispone de utilidades avanzadas de E/S, conversión de color y primitivas de dibujo.[8] **Inconvenientes:** instalación y dependencias más pesadas de lo necesario para una simulación inicial; curva de entrada mayor si sólo se requieren anotaciones simples. **Motivo de descarte:** se busca minimizar fricción y tiempo de puesta en marcha en el Sprint 1.

Matplotlib (Python). **Ventajas:** ampliamente conocida, permite renderizar y guardar imágenes con textos y formas.[3] **Inconvenientes:** está orientada a gráficos científicos, no al procesamiento de imágenes en un servicio web; el flujo para anotar y devolver imágenes resulta menos natural. **Motivo de descarte:** no optimiza el caso de uso de backend (anotar rápidamente y devolver).

Librería elegida: Pillow

Descripción. Pillow es el *fork* moderno de la Python Imaging Library (PIL). Facilita abrir, manipular y guardar imágenes, e incluye utilidades sencillas para dibujar rectángulos, textos y otras formas.[2]

Ventajas para este proyecto.

- **Ligereza y rapidez de integración:** instalación simple con `pip` y API directa para anotar imágenes.
- **Ajustada al caso de uso:** permite simular detecciones dibujando cajas y etiquetas sobre la imagen y devolverla junto con un JSON de “detecciones”.
- **Encaje con Flask:** fácil de integrar en endpoints que reciben ficheros y devuelven flujos binarios/imagen.
- **Evolución controlada:** cuando se integren modelos reales (YOLO, MediaPipe, etc.), se puede mantener la misma salida (imagen + JSON) sin cambios en el contrato.

Justificación de elección. Pillow ofrece exactamente lo necesario para simular resultados visuales y datos estructurados con un coste de adopción mínimo, evitando la complejidad innecesaria de alternativas más pesadas.

Funcionamiento en el proyecto. El endpoint del servidor (Flask) recibirá una imagen, generará detecciones sintéticas (clase, confianza, caja), anotará la imagen con Pillow y devolverá: (i) la imagen marcada y (ii) un JSON con la lista de detecciones. Este contrato será compatible con la futura sustitución de la simulación por modelos de IA reales.

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

Este capítulo recoge los aspectos más relevantes del desarrollo del TFG, no sólo desde el punto de vista técnico, sino también desde la lógica que ha guiado las decisiones adoptadas a lo largo del proyecto. No se persigue realizar una mera descripción de tecnologías ni una cronología de tareas, sino exponer cómo ha evolucionado el trabajo, por qué se eligieron determinados caminos de solución y de qué forma cada fase ha condicionado la siguiente.

El desarrollo ha pasado por dos grandes etapas. La primera estuvo orientada a la construcción de una demo funcional con la que validar la

viabilidad técnica del procesamiento de imágenes mediante modelos de inteligencia artificial. La segunda, ya centrada en la aplicación real, ha consistido en formalizar los requisitos del sistema, delimitar un producto mínimo viable, diseñar el modelo de datos, adaptar el entorno de desarrollo a Linux e iniciar la implementación de la base de datos sobre MariaDB.

Esta evolución resulta especialmente significativa porque muestra un proceso de trabajo razonado. Primero se verificó que la idea podía materializarse técnicamente; después, se abordó su transformación en una aplicación completa, con estructura, persistencia, reglas de negocio y una base sólida para futuras ampliaciones.

5.1. Enfoque general y ciclo de vida

El proyecto se ha desarrollado siguiendo un enfoque **iterativo e incremental**, priorizando la validación temprana de riesgos técnicos y la construcción progresiva de una base estable para la aplicación final. En lugar de comenzar directamente con una implementación completa, se optó por construir primero una demo que permitiera:

- Confirmar que la ejecución de modelos de visión por computador era viable en el entorno previsto.
- Verificar el flujo completo de trabajo, desde la subida de la imagen hasta la obtención del resultado.
- Detectar de forma temprana problemas habituales relacionados con rutas, dependencias, formatos de entrada y salida o gestión de ficheros generados.
- Establecer una estructura inicial del proyecto que pudiera crecer sin requerir reescrituras profundas desde las primeras iteraciones.

Este planteamiento permitió reducir incertidumbre en las fases iniciales y tomar decisiones posteriores con una base empírica. La demo no se concibió como una versión reducida del producto final, sino como un entorno de validación técnica. Una vez comprobado el funcionamiento del flujo principal, el proyecto pasó a una fase distinta, más centrada en el análisis, el diseño y la preparación de la aplicación real.

En consecuencia, el ciclo de vida no ha sido lineal, sino progresivo. Cada etapa ha cumplido una función concreta: primero, aprender y validar;

después, formalizar y estructurar; finalmente, preparar una implementación alineada con el alcance real del sistema.

5.2. Planteamiento inicial del problema

El objetivo general del TFG es desarrollar una aplicación web capaz de actuar como servidor para ejecutar algoritmos o modelos de inteligencia artificial sobre imágenes, devolviendo tanto resultados estructurados como evidencias visuales del procesamiento realizado. Desde el inicio se definieron varios principios que han guiado el desarrollo del proyecto y que siguen presentes en la evolución hacia la aplicación final.

- **Resultado visual como salida principal.** Además de la información estructurada en formato JSON, la salida más relevante para la validación del sistema es la imagen procesada y anotada.
- **Extensibilidad.** El sistema debe permitir incorporar nuevos modelos y nuevos modos de ejecución sin obligar a replantear su núcleo.
- **Separación de responsabilidades.** Debe diferenciarse claramente la API orientada a integración y pruebas de la interfaz web destinada a demostración y uso interactivo.
- **Evolución hacia pipelines.** La arquitectura debe dejar abierta la posibilidad de encadenar distintas etapas de procesamiento, de modo que la salida de una pueda alimentar a la siguiente.

Desde una perspectiva más amplia, el problema no se reducía a ejecutar un modelo de IA sobre una imagen. También era necesario plantear una solución capaz de evolucionar hacia un producto con control de acceso, diferenciación por planes, gestión de catálogo de IAs, trazabilidad de ejecuciones y una base de datos que reflejase correctamente las reglas de negocio del sistema.

5.3. Elección tecnológica: Flask y ecosistema Python

Para el backend se optó por **Python** y **Flask**. La elección de Python resultaba especialmente adecuada por su amplia implantación en ámbitos de visión por computador e inteligencia artificial, así como por la madurez

de su ecosistema para manipulación de imágenes, inferencia y tratamiento de datos.

Por su parte, Flask se eligió por su ligereza y flexibilidad. En una primera fase del proyecto resultaba importante trabajar con una herramienta que permitiera construir una API funcional con rapidez, sin imponer una estructura excesivamente rígida. Esta característica lo hacía especialmente apropiado para una etapa de experimentación y validación, en la que convenía comprobar flujos reales antes de consolidar la organización interna del sistema. Al mismo tiempo, Flask permite evolucionar hacia una estructura más modular mediante rutas organizadas, servicios diferenciados y separación entre la lógica HTTP y la lógica de negocio.

Además, el ecosistema de Python permitió integrar de forma directa varias herramientas esenciales para el proyecto:

- **OpenCV (cv2)** y **NumPy** para la carga, decodificación, manipulación y almacenamiento de imágenes.
- Librerías y modelos de visión por computador con documentación abundante y uso extendido.
- Un entorno suficientemente flexible para evolucionar hacia una aplicación web real con persistencia, autenticación y control de acceso.

La combinación de Flask con estas bibliotecas ofrecía, por tanto, un equilibrio adecuado entre rapidez de desarrollo, claridad arquitectónica y compatibilidad con las necesidades técnicas del proyecto.

5.4. Selección de IAs para la demo: por qué YOLO y MediaPipe

Para la demo inicial se eligieron dos familias de modelos complementarias: **YOLO** y **MediaPipe**. La selección respondió a la necesidad de validar distintos tipos de procesamiento sobre imágenes y de obtener salidas visuales lo suficientemente expresivas como para comprobar de manera inmediata el correcto funcionamiento del sistema.

YOLO: detección general de objetos

YOLO se seleccionó por ser uno de los modelos de detección de objetos más extendidos en la práctica. Su principal ventaja en el contexto de una

demo es que produce una salida muy fácil de interpretar: cajas delimitadoras sobre los objetos detectados, acompañadas de una etiqueta y un nivel de confianza. Esto permitía verificar rápidamente tanto la correcta ejecución del modelo como la capacidad del sistema para generar una imagen anotada con valor demostrativo.

Además, YOLO aportaba una validación útil desde el punto de vista funcional, ya que representa un tipo de análisis generalista aplicable a escenarios muy diversos. Su inclusión en la demo permitía mostrar que la aplicación no estaba restringida a un único caso de uso especialmente específico.

MediaPipe: detección de landmarks (pose y manos)

MediaPipe se incorporó porque aportaba una funcionalidad distinta y complementaria a la detección por cajas. Mientras que YOLO trabaja a nivel de objeto, MediaPipe permite operar con **landmarks**, es decir, con puntos clave anatómicos y sus conexiones sobre la imagen. Esta salida posee un alto valor visual y resulta muy útil para demostrar que el sistema puede manejar no sólo detecciones discretas, sino también estructuras geométricas más ricas.

En la demo se trabajó concretamente con dos modos:

- **Pose.** Detección de landmarks corporales y sus conexiones.
- **Hands.** Detección de landmarks de manos y relaciones anatómicas.

La combinación de YOLO y MediaPipe permitió representar dos paradigmas frecuentes dentro de la visión por computador:

- Detección a nivel de objeto.
- Análisis a nivel de estructura o puntos clave.

De este modo, la demo no sólo validó que el sistema podía ejecutar modelos de IA, sino también que era capaz de integrar salidas de naturaleza distinta dentro de un mismo flujo de uso.

5.5. Construcción de la demo: objetivos y aprendizajes clave

Motivación de la demo

Antes de iniciar la aplicación final, se desarrolló una demo con dos objetivos principales:

1. **Entender el flujo técnico real.** Cómo recibe el servidor una imagen, cómo se transforma a una estructura adecuada para el modelo, cómo se procesan los resultados y cómo se generan evidencias visuales.
2. **Validar decisiones de arquitectura.** Comprobar que una organización basada en tareas por modelo y modo, junto con una factoría para seleccionarlas, constituía una base razonable para el crecimiento posterior.

La demo no se concibió, por tanto, como una versión reducida de la aplicación definitiva, sino como un entorno de aprendizaje controlado. Su valor principal residía en reducir incertidumbre y permitir que las siguientes decisiones se apoyaran en problemas ya observados en la práctica.

Resultado de la demo (visión general)

En esta fase se consiguió implementar un flujo completo de extremo a extremo:

- Subida de una imagen desde el cliente.
- Ejecución del modelo seleccionado con el modo correspondiente.
- Devolución de una salida estructurada con información básica del procesamiento.
- Generación y almacenamiento de una imagen anotada en disco.

Este resultado permitió verificar varios aspectos esenciales: la viabilidad del procesamiento, el correcto funcionamiento de la comunicación cliente-servidor, la recuperación de la imagen generada desde la interfaz web y la posibilidad de integrar distintos modelos bajo una misma lógica de uso.

5.5. CONSTRUCCIÓN DE LA DEMO: OBJETIVOS Y APRENDIZAJES CLAVE

21

Para documentar el funcionamiento de la demo, se incorporan a continuación varias capturas representativas de la interfaz y de los resultados obtenidos con los distintos modelos y modos implementados.

En primer lugar, la Figura 5.1 muestra el funcionamiento de MediaPipe en el modo **hands**. En este caso, la salida visual representa landmarks y conexiones anatómicas sobre las manos detectadas, lo que permite ilustrar de forma clara un análisis basado en puntos clave y no únicamente en detecciones por caja.

Demo IA del TFG

Subir imagen:

Seleccionar archivo

Ningún archivo seleccionado

Modelo

Mediapipe

Modo

Hands

Analizar

Resultado



```
{
  "mode": "hands",
  "model": "mediapipe",
  "num_hands": 2,
  "output_image": "outputs\\mediapipe_hands_4f1d25047baf4b218bafc6e624de3cde.png"
}
```

A continuación, la Figura 5.2 presenta una ejecución de MediaPipe en el modo **pose**. Esta captura permite comprobar cómo la misma interfaz puede reutilizarse para distintos modelos y modos, manteniendo un flujo de uso homogéneo desde el punto de vista del usuario.

Demo IA del TFG

Subir imagen:

Seleccionar archivo

Ningún archivo seleccionado

Modelo

Mediapipe

Modo

Pose

Analizar

Resultado



Figura 5.2: Interfaz de la demo durante una ejecución de MediaPipe en modo **pose**, mostrando la imagen de entrada y los landmarks corporales detectados.

La Figura 5.3 recoge parte de la salida estructurada correspondiente al modo **pose**. En este caso, el sistema devuelve la información asociada a los landmarks detectados, incluyendo visibilidad, coordenadas y referencia a la imagen de salida anotada.

```
{
  "visibility": 0.28028222918510437,
  "x": 0.21716925501823425,
  "y": 1.1368986368179321,
  "z": -1.350184440612793
},
{
  "visibility": 0.00883315410465002,
  "x": 0.7074923515319824,
  "y": 1.174790620803833,
  "z": 0.0024941624142229557
},
{
  "visibility": 0.009900550357997417,
  "x": 0.2907187342643738,
  "y": 1.1703097820281982,
  "z": 0.0039550870711791515
},
{
  "visibility": 0.0032810361590236425,
  "x": 0.6867023706436157,
  "y": 1.612317681312561,
  "z": 0.12653392553329468
},
{
  "visibility": 0.0010673474753275514,
  "x": 0.2965492904186249,
  "y": 1.6022087335586548,
  "z": -0.010423111729323864
},
{
  "visibility": 0.00021656697208527476,
  "x": 0.6710666418075562,
  "y": 1.9937782287597656,
  "z": 1.034913420677185
},
{
  "visibility": 5.2098224841756746e-05,
  "x": 0.30643802881240845,
  "y": 1.9819284677505493,
  "z": 0.6339143514633179
},
{
  "visibility": 0.0001776724384399131,
  "x": 0.6762940883636475,
  "y": 2.0606932640075684,
  "z": 1.0026290845870972
},
{
  "visibility": 0.00010073753946926445,
  "x": 0.30443406105041504,
  "y": 2.048147201538086,
  "z": 0.6703062057495117
},
{
  "visibility": 0.00023552048142571747,
  "x": 0.6121371388435364,
  "y": 2.122279167175293,
  "z": 0.33695098757743835
},
{
  "visibility": 0.00012527908256743103,
  "x": 0.36167168617248535,
  "y": 2.10892438885498,
  "z": -0.16412502527236938
}
},
"mode": "pose",
"model": "mediapipe",
"num_landmarks": 33,
"output_image": "outputs\\mediapipe_pose_098757753f3c4d1999a5880fa45e457b.png"
}
```

Figura 5.3: Salida JSON generada por MediaPipe en modo **pose**, con información de landmarks, visibilidad y ruta de la imagen procesada.

Una vez validado el comportamiento de MediaPipe, se incorporó también YOLO como modelo de detección general de objetos. La Figura 5.4 muestra la interfaz de la demo tras seleccionar una imagen y ejecutar el modelo YOLO en su modo por defecto. Puede observarse tanto el formulario de entrada como la imagen anotada devuelta al usuario.

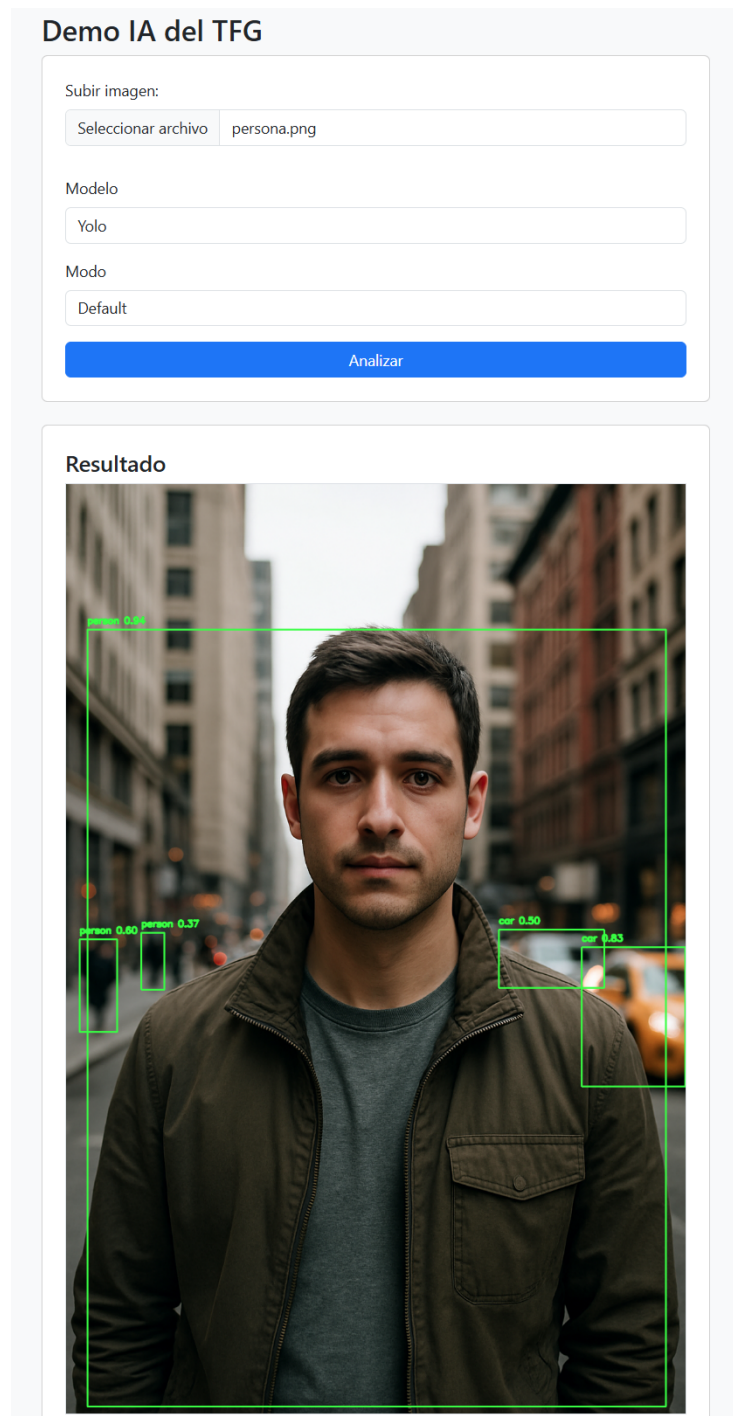


Figura 5.4: Interfaz de la demo con ejecución del modelo YOLO en modo `default`, mostrando la imagen procesada con cajas delimitadoras sobre los objetos detectados.

5.5. CONSTRUCCIÓN DE LA DEMO: OBJETIVOS Y APRENDIZAJES CLAVE

27

Por su parte, la Figura 5.5 presenta el detalle de la salida estructurada generada para esa misma ejecución. En ella se aprecia la lista de detecciones, incluyendo etiquetas, niveles de confianza y coordenadas de las cajas asociadas a cada objeto identificado.

```
{
  "detections": [
    {
      "bbox": {
        "xmax": 991.740478515625,
        "xmin": 35.5536003112793,
        "ymax": 1524.26513671875,
        "ymin": 240.70657348632812
      },
      "confidence": 0.9417006969451904,
      "label": "person"
    },
    {
      "bbox": {
        "xmax": 1023.9545288085938,
        "xmin": 852.6067504882812,
        "ymax": 995.4043579101562,
        "ymin": 765.3660278320312
      },
      "confidence": 0.829071581363678,
      "label": "car"
    },
    {
      "bbox": {
        "xmax": 84.07442474365234,
        "xmin": 22.836565017700195,
        "ymax": 905.7379760742188,
        "ymin": 752.4883422851562
      },
      "confidence": 0.5996273756027222,
      "label": "person"
    },
    {
      "bbox": {
        "xmax": 889.6104736328125,
        "xmin": 715.0587768554688,
        "ymax": 832.171142578125,
        "ymin": 736.8294677734375
      },
      "confidence": 0.49799877405166626,
      "label": "car"
    },
    {
      "bbox": {
        "xmax": 162.29664611816406,
        "xmin": 124.18800354003906,
        "ymax": 835.3543701171875,
        "ymin": 741.5029907226562
      },
      "confidence": 0.370271235704422,
      "label": "person"
    }
  ],
  "mode": "default",
  "model": "yolo",
  "output_image": "outputs\\yolo_default_24955d766b514a1bb23a2ade29c99d01.png"
}
```

Figura 5.5: Salida JSON asociada a la ejecución de YOLO, incluyendo detecciones, confianza, coordenadas y ruta de la imagen generada.

5.6. Decisiones de diseño relevantes en la demo

Arquitectura basada en tareas y patrón factory

Uno de los aspectos más relevantes de la fase inicial fue estructurar la lógica de IA como un conjunto de tareas especializadas y una factoría central encargada de seleccionar cuál debía ejecutarse en cada caso.

- Cada modelo dispone de un bloque de lógica propio que gestiona sus modos de funcionamiento.
- La factoría global decide qué tarea devolver a partir de los parámetros recibidos, como el modelo y el modo seleccionados.

Esta decisión permitió evitar condicionales dispersos en las rutas del servidor y favoreció una organización más limpia y extensible. En lugar de mezclar la lógica de selección del modelo con la capa HTTP o con el tratamiento de imágenes, cada responsabilidad quedó separada. Esta estructura resultó especialmente útil porque anticipaba el crecimiento del proyecto: añadir un nuevo modo, o incluso un nuevo modelo, pasaba a ser una operación localizada y no una modificación repartida por todo el sistema.

Separación entre API REST e interfaz web

Durante el desarrollo se identificó muy pronto la conveniencia de separar claramente la parte orientada a integración de la parte orientada a interacción humana.

- La **API REST**, bajo un prefijo específico, se reservó para pruebas, integración y consumo programático.
- La **Interfaz web**, situada en la raíz de la aplicación, se mantuvo como punto de acceso visual y demostrativo.

Esta separación evitó mezclar respuestas JSON con renderizado HTML y facilitó mantener una lógica de servidor más ordenada. Además, resultó coherente con la evolución posterior del sistema, ya que la aplicación real deberá sostener tanto una capa de presentación como servicios internos claramente diferenciados.

Gestión de rutas y entrega de ficheros generados

Uno de los aprendizajes más significativos de la demo estuvo relacionado con la gestión de rutas y la entrega de imágenes generadas al navegador. Las tareas guardaban los resultados en un directorio de salida y, para hacerlos accesibles desde la interfaz, fue necesario resolver correctamente varios aspectos:

- Creación de una ruta específica para servir los ficheros generados.
- Gestión adecuada del directorio de trabajo y de la raíz real de la aplicación.
- Tratamiento correcto de rutas relativas y absolutas.
- Control de los separadores de ruta en distintos sistemas operativos.

Aunque pudiera parecer un detalle secundario, este punto fue decisivo para convertir una prueba local en una demo realmente utilizable. Además, permitió anticipar uno de los problemas que la aplicación final tendría que resolver con mayor formalidad: la gestión coherente de entradas, salidas y ficheros temporales en un entorno web.

Interfaz dinámica según modelos y modos disponibles

A medida que la demo incorporó distintos modelos y modos, se hizo evidente que una interfaz estática no escalaba adecuadamente. Por ello, se diseñó una interacción más dinámica entre frontend y backend:

- El selector de modelos se rellena en función de la configuración disponible en el servidor.
- El selector de modos depende del modelo elegido, evitando mostrar opciones no aplicables.

Esta decisión mejora la experiencia de uso y reduce el acoplamiento entre la interfaz y la lógica del servidor. Además, anticipa una necesidad propia de la aplicación real: que la interfaz refleje únicamente las opciones efectivamente disponibles y autorizadas en cada contexto.

5.7. Aspectos relevantes de implementación en la demo

Representación de imágenes y compatibilidad con los modelos

Durante la demo se consolidó un flujo interno consistente para el tratamiento de imágenes:

- Recepción de la imagen como parte de un formulario multipart.
- Decodificación de los bytes a una matriz de NumPy.
- Procesamiento mediante el modelo correspondiente.
- Anotación directa sobre la propia imagen en memoria.
- Almacenamiento del resultado procesado en disco.

La utilización de OpenCV y NumPy permitió homogeneizar la representación interna de la imagen, algo especialmente útil para conectar modelos distintos bajo una lógica común. Esta homogeneidad también resulta clave de cara a una futura evolución hacia pipelines, donde la salida de una etapa podrá transformarse en entrada de la siguiente.

Salida dual: JSON y evidencia visual

Se estableció una convención de salida que combina dos tipos de resultado:

- Una respuesta JSON con información estructurada de alto nivel.
- Una referencia a la imagen procesada generada.

Esta decisión posee un valor especial en un contexto académico y demostrativo. El JSON aporta trazabilidad, estructuración y posibilidades de integración, mientras que la imagen anotada proporciona una evidencia visual inmediata del resultado obtenido. La combinación de ambos elementos refuerza la capacidad de validación del sistema y facilita el análisis de su comportamiento.

5.8. Limitaciones conocidas de la demo y necesidad de evolución

La demo cumplió su objetivo principal: validar la viabilidad técnica del flujo de procesamiento y establecer una arquitectura inicial extensible. Sin embargo, seguía siendo un prototipo con limitaciones deliberadas.

- **Persistencia reducida.** Los resultados se gestionaban como ficheros y rutas, sin un modelo de datos completo detrás.
- **Ausencia de gestión de usuarios y permisos.** La demo no incorporaba roles, planes ni control de acceso.
- **Trazabilidad limitada.** No existía aún un registro persistente y estructurado de ejecuciones, configuraciones y resultados.
- **Ejecución aislada.** Cada tarea se ejecutaba de forma independiente, sin cadenas configurables de procesamiento.
- **Robustez mejorable.** El tratamiento de errores, la validación de entradas y la organización general todavía debían evolucionar para acercarse a una aplicación real.

En definitiva, la demo actuó como un **escalón de aprendizaje y validación**. Su principal aportación no fue únicamente producir un prototipo funcional, sino permitir identificar qué decisiones podían mantenerse en la aplicación final y cuáles debían replantearse desde una perspectiva más amplia. A partir de este punto, el proyecto dejó de centrarse exclusivamente en demostrar la viabilidad técnica del procesamiento y pasó a abordarse como una aplicación completa.

5.9. De la demo a la aplicación real: cambio de enfoque

Una vez validada la viabilidad técnica mediante la demo, el proyecto entró en una nueva fase con un objetivo diferente. Hasta ese momento, el trabajo había estado orientado sobre todo a comprobar que era posible recibir una imagen, procesarla con distintos modelos y devolver una salida útil. Sin embargo, demostrar que algo funciona no equivale todavía a disponer de una aplicación completa, coherente y preparada para evolucionar.

Por ello, se decidió no seguir ampliando la demo de forma improvisada, sino iniciar la construcción de la aplicación real a partir de un proceso más formal de análisis y diseño. Esta transición constituye uno de los hitos principales del proyecto, ya que supone pasar de un prototipo técnico a un sistema software definido desde las necesidades del producto, las restricciones del dominio y los criterios de crecimiento futuro.

La demo había permitido responder a una primera cuestión: si el procesamiento de imágenes mediante IA era viable dentro del proyecto. La nueva fase debía responder a otra diferente: cómo convertir esa viabilidad en una aplicación mantenible, ampliable y alineada con los objetivos reales del TFG.

5.10. Análisis de requisitos de la aplicación completa

El primer paso de esta nueva etapa consistió en definir los **requisitos funcionales**, los **requisitos no funcionales** y los **casos de uso** de la aplicación completa. Esta decisión resultó especialmente importante, ya que permitió comenzar la aplicación real desde una visión global del sistema y no desde un conjunto disperso de funcionalidades aisladas.

Se optó por describir primero el sistema completo porque diseñar directamente a partir del MVP habría implicado un riesgo considerable: construir una solución válida a corto plazo, pero débil desde el punto de vista estructural. En cambio, partir de una visión global permitió identificar desde el principio los elementos esenciales del dominio, aunque algunos de ellos no fueran a implementarse por completo en la primera iteración.

Entre los aspectos definidos en esta fase destacan:

- La existencia de distintos tipos de usuario y roles dentro del sistema.
- La diferenciación entre planes de acceso y permisos efectivos sobre los recursos.
- La gestión del catálogo de modelos de IA y de sus distintos modos de ejecución.
- La necesidad de registrar ejecuciones y resultados de forma estructurada.

5.11. DEL SISTEMA COMPLETO AL PRODUCTO MÍNIMO VIABLE

- La previsión de pipelines como evolución natural del sistema.
- La necesidad de incorporar funciones de administración y mantenimiento.

Desde el punto de vista funcional, esta fase permitió concretar capacidades como la autenticación, el control de acceso, la selección de modelos y modos disponibles, la ejecución de inferencias, la gestión de resultados y la administración del catálogo. Desde el punto de vista no funcional, ayudó a fijar criterios de modularidad, extensibilidad, mantenibilidad, seguridad, robustez y cercanía a un futuro despliegue real.

Por tanto, esta fase de análisis no constituyó un mero trámite documental, sino una base necesaria para que el resto del desarrollo tuviera coherencia interna.

5.11. Del sistema completo al producto mínimo viable

Una vez establecida la visión global, el siguiente paso fue derivar de ella el **producto mínimo viable (MVP)**. Esta forma de proceder resultó especialmente útil porque el MVP no se definió como un conjunto arbitrario de funcionalidades mínimas, sino como un subconjunto coherente del sistema final.

El valor del MVP, en este contexto, no residía simplemente en hacer una versión más pequeña, sino en seleccionar aquellas capacidades imprescindibles para demostrar el funcionamiento del núcleo del producto. En este proyecto, ello implicaba poder recorrer el flujo principal de valor de la aplicación: acceso al sistema, selección de una IA autorizada, envío de una imagen, ejecución del procesamiento y obtención del resultado.

De este modo, el MVP se concibió como la primera versión útil de la aplicación real y no como una demo independiente. Esta decisión aportó varias ventajas:

- Evitó sobredimensionar la primera implementación de la aplicación.
- Permitted conservar coherencia con la arquitectura prevista a medio plazo.
- Redujo retrabajos posteriores.

- Facilitó que el diseño de datos y la organización de la lógica no tuvieran que rehacerse al crecer el sistema.

La definición del MVP a partir del sistema completo fue, por tanto, una decisión de diseño relevante. Gracias a ella, la primera iteración de la aplicación real pudo centrarse en lo esencial sin perder coherencia con el alcance general del proyecto.

5.12. De los casos de uso al diseño de datos

Una vez definidos los requisitos y los casos de uso, el trabajo avanzó hacia el diseño de la persistencia. Esta transición fue natural, ya que los casos de uso permitían identificar qué información debía existir en el sistema, qué relaciones unían a las distintas entidades y qué restricciones debían preservarse para mantener la coherencia del dominio.

A partir de este análisis se elaboraron, en primer lugar, el **diagrama Entidad-Relación** y, posteriormente, el **modelo relacional**. Su construcción no se abordó como un ejercicio meramente académico, sino como un paso necesario para traducir el comportamiento esperado de la aplicación a una estructura de datos robusta.

En esta fase el proyecto dejó de apoyarse exclusivamente en el procesamiento puntual de imágenes para incorporar una dimensión claramente orientada a producto. El sistema pasó a necesitar representar entidades como usuarios, roles, planes, suscripciones, modelos IA, modos, pipelines y ejecuciones. Es decir, ya no bastaba con que el procesamiento funcionara; era necesario que la lógica de acceso, configuración y trazabilidad quedara recogida en un modelo persistente coherente.

Criterios seguidos en el modelado

Durante la elaboración de los diagramas se siguieron varios criterios de diseño especialmente relevantes:

- **Separación clara de conceptos.** Cada tabla debía representar una entidad o relación con significado propio, evitando mezclar información heterogénea.
- **Reducción de redundancia.** Se procuró minimizar duplicidades que pudieran generar inconsistencias, especialmente en todo lo relativo a permisos, tipos de plan y catálogo de IAs.

- **Preparación para el crecimiento.** Aunque el MVP no incorpora toda la funcionalidad futura, el modelo se diseñó teniendo en cuenta la incorporación de nuevas IAs, nuevos modos y futuras cadenas de procesamiento.
- **Trazabilidad.** Se concedió importancia a conservar información histórica relevante, especialmente en las ejecuciones y en su relación con configuraciones activas en cada momento.
- **Integridad de negocio.** El diseño se apoyó en claves primarias, claves foráneas, restricciones de unicidad y otras limitaciones que trasladan parte de las reglas del dominio al propio esquema de datos.

Este proceso permitió además aclarar varios conceptos que, aunque próximos entre sí, debían mantenerse separados para conservar la flexibilidad del sistema. Por ejemplo, un usuario no es equivalente a su plan, una IA no es lo mismo que uno de sus modos, y una ejecución concreta debe poder conservarse incluso aunque determinados elementos configurables del sistema cambien con el tiempo.

Valor del diagrama E-R y del modelo relacional

La utilidad de estos diagramas fue doble. Por un lado, actuaron como documentación de diseño. Por otro, sirvieron como herramienta de depuración conceptual, ya que obligaron a concretar relaciones, cardinalidades y dependencias antes de llegar al nivel de SQL o de implementación.

Disponer del modelo relacional previamente definido facilitó notablemente la fase posterior de implementación en MariaDB. En lugar de crear tablas de forma improvisada conforme surgían necesidades puntuales, se partió de un esquema global ya razonado, lo que mejoró la consistencia entre análisis, diseño e implementación.

5.13. Migración del entorno de desarrollo a Debian

Una vez estabilizado el diseño lógico del sistema, se tomó otra decisión relevante en el desarrollo del proyecto: la migración del entorno de trabajo a **Linux, concretamente Debian**. Esta elección no respondió a una preferencia casual, sino a razones técnicas relacionadas con la propia naturaleza del proyecto y con su posible despliegue posterior.

Durante la fase de demo se detectaron varios aspectos del entorno que condicionaban el trabajo diario: diferencias en el tratamiento de rutas, dependencia del sistema operativo, gestión desigual de permisos y cierta distancia entre el entorno local de desarrollo y el tipo de plataforma sobre la que previsiblemente se ejecutará la aplicación en un escenario real. Ante esta situación, resultaba coherente trasladar el desarrollo a un sistema más cercano al destino natural de una aplicación web de estas características.

Justificación técnica del cambio

La elección de Debian se apoyó en varios motivos principales.

En primer lugar, **la cercanía al entorno de despliegue**. La mayor parte de los servidores donde se alojan aplicaciones web y bases de datos trabajan sobre distribuciones Linux o entornos equivalentes. Desarrollar directamente sobre Debian reduce la distancia entre desarrollo y producción y disminuye la probabilidad de encontrar problemas tardíos derivados del sistema operativo.

En segundo lugar, **la naturalidad con la que encajan en Linux las tecnologías utilizadas**. Python, Flask, MariaDB y la mayor parte de utilidades de backend, administración y automatización se integran de forma especialmente cómoda en este entorno. Esto simplifica la instalación de herramientas, la configuración de servicios y la organización del proyecto.

En tercer lugar, **la consistencia en el tratamiento de rutas, permisos y procesos**. Una aplicación que debe gestionar ficheros temporales, imágenes generadas y servicios de backend se beneficia de un entorno donde estos aspectos se comportan de forma predecible y cercana a la realidad del servidor final. El modelo de permisos y la organización del sistema de ficheros de Linux resultan especialmente apropiados en este contexto.

En cuarto lugar, **la facilidad de automatización y administración**. Debian permite trabajar con comodidad mediante scripts, variables de entorno, herramientas de consola y servicios del sistema. Esta capacidad resulta útil no sólo durante el desarrollo, sino también de cara a futuras fases de despliegue, monitorización o integración continua.

Por último, también se valoró la **estabilidad del sistema**. Debian es una distribución ampliamente utilizada en entornos de servidor por su robustez, su madurez y su equilibrio entre estabilidad y compatibilidad. En un proyecto académico en el que se busca justificar técnicamente las decisiones adoptadas, esta elección resulta especialmente sólida.

Ventajas concretas para el proyecto

En el contexto de este TFG, el paso a Debian aportó beneficios claros:

- Un entorno más adecuado para desarrollar servicios web y bases de datos.
- Una gestión más natural de dependencias, permisos y rutas.
- Mayor cercanía a escenarios reales de despliegue.
- Mejor preparación para herramientas habituales de backend y administración.
- Menor fricción al integrar tecnologías que, por su naturaleza, suelen ejecutarse sobre Linux.

Por tanto, la migración a Debian debe entenderse como una decisión de ingeniería orientada a mejorar la coherencia del proyecto y no como un cambio accesorio dentro del proceso.

5.14. Implementación de la base de datos en MariaDB

Con el modelo conceptual y relacional ya definidos, y una vez consolidado el nuevo entorno de desarrollo en Debian, se procedió a la implementación de la base de datos en **MariaDB**. Esta fase supuso el paso desde el diseño abstracto a una persistencia real y operativa sobre la que podrá apoyarse la lógica de la aplicación.

La elección de MariaDB resultó adecuada por varios motivos. En primer lugar, se trata de un sistema gestor maduro, ampliamente utilizado y plenamente válido para una aplicación web de estas características. En segundo lugar, su integración en entornos Linux es muy natural, lo que reforzaba la coherencia del cambio de sistema operativo realizado previamente. Además, proporciona soporte suficiente para modelar correctamente las relaciones del sistema mediante claves foráneas, restricciones, índices y comportamiento transaccional.

La implementación del esquema no consistió únicamente en traducir entidades a tablas. Fue también el momento de trasladar al nivel físico varias decisiones adoptadas durante el diseño:

- Definición de claves primarias para garantizar una identificación inequívoca.
- Creación de restricciones **UNIQUE** para impedir duplicidades indebidas.
- Establecimiento de claves foráneas que reflejan las relaciones del dominio.
- Incorporación de valores por defecto y restricciones que ayudan a preservar la coherencia de estados y fechas.
- Selección de un motor transaccional adecuado para mantener la integridad de las operaciones.

Desde el punto de vista del proyecto, esta fase fue especialmente significativa porque permitió materializar por primera vez la estructura persistente de la aplicación real. A partir de aquí, el sistema deja de depender únicamente de la lógica de procesamiento heredada de la demo y comienza a disponer de una base sólida para gestionar usuarios, permisos, configuraciones y ejecuciones de forma consistente.

Además, la base de datos no se planteó únicamente como un repositorio de información, sino como un elemento capaz de reflejar correctamente el comportamiento del sistema. Esto llevó a prestar especial atención a decisiones como la separación entre catálogo y uso real, la preservación de la trazabilidad y el tratamiento cuidadoso de determinadas relaciones para no comprometer información histórica relevante.

5.15. Transición hacia las siguientes fases del desarrollo

Con la definición de requisitos, la delimitación del MVP, el diseño del modelo de datos, la migración del entorno a Debian y la implementación inicial de la base de datos en MariaDB, el proyecto deja atrás la etapa centrada exclusivamente en la validación técnica de la demo y pasa a apoyarse en una base más sólida para el desarrollo de la aplicación real. A partir de este punto, las siguientes iteraciones se orientarán a incorporar de forma progresiva los distintos componentes funcionales del sistema, manteniendo la coherencia con las decisiones de análisis y diseño adoptadas en esta fase.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] Zachary J Bortolot and Randolph H Wynne. Estimating forest biomass using small footprint lidar data: An individual tree-based approach that incorporates training data. *ISPRS Journal of Photogrammetry and Remote Sensing*, 59(6):342–360, 2005.
- [2] Pillow Contributors. Pillow (pil fork) documentation, 2025. Consultado el 11 de noviembre de 2025.
- [3] Matplotlib Developers. Matplotlib documentation, 2025. Consultado el 11 de noviembre de 2025.
- [4] Google AI Edge. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. Consultado en noviembre de 2025.
- [5] Google AI Edge. Repositorio oficial mediapipe en github. <https://github.com/google-ai-edge/mediapipe>, 2024. Consultado en noviembre de 2025.
- [6] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- [7] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Eric Uboweja, Marcus Hays, Fan Zhang, Chuo-Ling Chang, Michael G Yong, Jiyong Lee, Wei-Ting Chang, Wan-Te Hua, Marion Georg, and Matthias Grundmann. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.
- [8] OpenCV Team. Opencv documentation, 2025. Consultado el 11 de noviembre de 2025.

- [9] Ultralytics. Repositorio oficial yolo en github. <https://github.com/ultralytics/ultralytics>, 2024. Consultado en noviembre de 2025.
- [10] Ultralytics. YOLOv8 documentation. <https://docs.ultralytics.com/models/yolov8/>, 2024. Consultado en noviembre de 2025.
- [11] Wikipedia. Latex — wikipedia, la enciclopedia libre. <https://es.wikipedia.org/w/index.php?title=LaTeX&oldid=84209252>, 2015. [Internet; descargado 30-septiembre-2015].